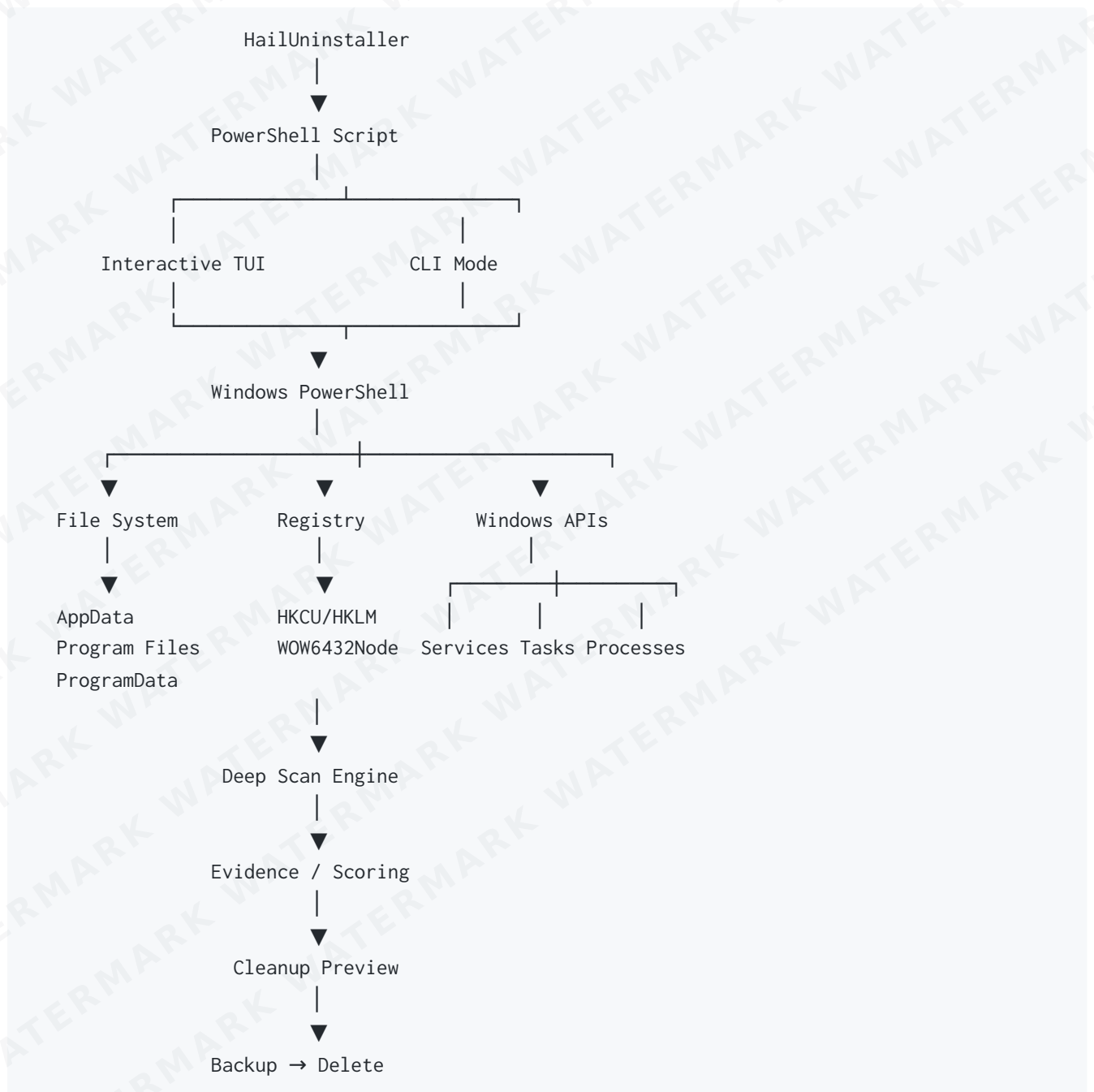


HailUninstaller to follow the same philosophy as **Win11Debloat**: a lightweight, **no-installation PowerShell** project that the user can launch with one command, gets an interactive terminal UI, performs the work, and exits. Win11Debloat itself uses a PowerShell script, supports a `Run.bat`, and can be launched directly from a PowerShell one-liner. [GitHub](#)

So I would **change my previous recommendation**: don't use Rust/Tauri/React for v1.

HailUninstaller architecture



The user experience

The user could run:

```
irm https://hailuninstaller.dev | iex
```

and get:

```
HAILUNINSTALLER
Deep Application Removal Tool
```

1. Installed Applications
2. Scan for Leftovers
3. Deep Scan
4. Cleanup History
5. Restore
- Q. Exit

Select an option:

That's **exactly the type of project you're describing.**

The important difference from Win11Debloat

Win11Debloat primarily modifies/removes Windows components and settings. Its documented features include app removal, privacy settings, system changes, Windows Update settings, Start/Search, Taskbar, Explorer, etc. [GitHub](#)

HailUninstaller would have one focused purpose:

Completely identify and remove an application's remaining footprint.

So don't make it a Windows debloater.

HailUninstaller modules

I'd make the repository like this:

```
HailUninstaller/
|
├── HailUninstaller.ps1
├── Run.bat
└── LICENSE
```

```
├── README.md
├── Config/
│   ├── Applications.json
│   ├── ProtectedPaths.json
│   └── Rules.json
├── Modules/
│   ├── UI.ps1
│   ├── Applications.ps1
│   ├── Scanner.ps1
│   ├── FileScanner.ps1
│   ├── RegistryScanner.ps1
│   ├── ServiceScanner.ps1
│   ├── TaskScanner.ps1
│   ├── ProcessScanner.ps1
│   ├── ShortcutScanner.ps1
│   ├── Evidence.ps1
│   ├── Cleanup.ps1
│   ├── Backup.ps1
│   └── Restore.ps1
└── Tests/
    ├── Scanner.Tests.ps1
    ├── Registry.Tests.ps1
    └── Cleanup.Tests.ps1
```

This is very similar in spirit to how Win11Debloat is organized into a main PowerShell script plus supporting folders/configuration/scripts. [GitHub](#)

Main workflow

Step 1 – Detect installed applications

```
Windows Registry
+
MSI
+
MSIX/AppX
+
known uninstallers
↓
Application List
```

Display:

Installed Applications

> Google Chrome

Discord

VLC

Blender

VS Code

Spotify

Search: _____

Step 2 – Select application

Google Chrome

Version: 154.x

Publisher: Google LLC

Location: C:\Program Files\Google\Chrome

Then:

[1] Normal Uninstall

[2] Uninstall + Deep Scan

[3] Scan Only

[4] Cancel

Step 3 – Normal uninstall

HailUninstaller should **first use the application's own uninstaller**.

Don't immediately delete its files.

HailUninstaller

↓

Find official uninstall command

↓

Run it

↓

Wait for completion

↓

Refresh system state

↓
Deep scan

This is safer and more compatible.

Step 4 — Deep scan

Now the interesting part.

Scanning:

[██] 100%

- ✓ Program Files
- ✓ Program Files (x86)
- ✓ AppData\Local
- ✓ AppData\Roaming
- ✓ ProgramData
- ✓ Registry
- ✓ Services
- ✓ Scheduled Tasks
- ✓ Startup
- ✓ Shortcuts
- ✓ Temporary data

Step 5 — Evidence engine

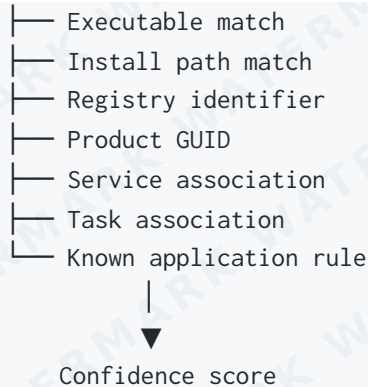
This is what will make HailUninstaller different.

Don't do:

```
if ($path -like "*Chrome*") {  
    Remove-Item  
}
```

Instead:

```
Candidate  
|  
├─ Application name match  
├─ Publisher match
```



Example:

- 98% C:\Users\...\AppData\Local\Discord
- 95% C:\ProgramData\Discord
- 94% DiscordUpdater service
- 71% HKCU\Software\Discord
- 62% C:\Users\...\AppData\Roaming\SomeSharedFolder

Step 6 — Preview before deletion

This is **mandatory**.



HIGH CONFIDENCE

[x] AppData\Local\Discord	742 MB
[x] AppData\Roaming\Discord	83 MB
[x] ProgramData\Discord	21 MB

REVIEW

[] HKCU\Software\Discord	Registry
[] DiscordUpdater	Service

PROTECTED

- [-] Shared Microsoft component
- [-] Windows system file

Selected: 846 MB

[A] Select All
[N] Select None
[R] Review
[D] Delete Selected
[B] Back

Step 7 — Backup

Before deleting:

Create cleanup backup?

[Y] Yes
[N] No

Store something like:

```
%LOCALAPPDATA%\HailUninstaller\  
Backups\  
  2026-10-02_Discord\  
    files\  
    registry\  
    manifest.json
```

Then:

Backup complete.

846 MB ready for removal.

Continue? [Y/N]

Step 8 — Cleanup

Then:

Removing:

✓ Files

- ✓ Folders
- ✓ Registry entries
- ✓ Services
- ✓ Scheduled tasks
- ✓ Shortcuts

 100%

Cleanup complete.

Removed: 846 MB

Items: 37

Press Enter to continue...

Step 9 — Restore

This is another feature I'd definitely include:

Cleanup History

- | | | |
|------------|--------|-------------|
| 1. Discord | 846 MB | 02 Oct 2026 |
| 2. Blender | 1.2 GB | 30 Sep 2026 |
| 3. VLC | 92 MB | 29 Sep 2026 |

[R] Restore

[D] Delete history

[B] Back

CLI mode too

Just like Win11Debloat supports both an interactive interface and CLI parameters, HailUninstaller should support both. [GitHub](#)

For example:

```
HailUninstaller.ps1 -List
```

```
HailUninstaller.ps1 -Scan "Discord"
```

```
HailUninstaller.ps1 -DeepScan "Discord"
```

```
HailUninstaller.ps1 -Uninstall "Discord"
```

And eventually:

```
HailUninstaller.ps1 -Uninstall "Discord" -DeepClean -CreateBackup
```

One-command distribution

This is the part you were talking about.

Win11Debloat uses a remote PowerShell bootstrap command to download and launch its script, with temporary files cleaned up afterward. [GitHub](#)

HailUninstaller can use the same model:

```
& ([scriptblock]::Create((irm "https://hailuninstaller.dev/")))
```

or, if hosted directly from GitHub:

```
irm https://raw.githubusercontent.com/YOURNAME/HailUninstaller/main/HailUninstaller.ps1 | iex
```

You can also provide:

```
Run.bat
```

for users who download the repository, just like Win11Debloat does. [GitHub](#)

So the final technology decision

For **your exact idea**:

- ✓ PowerShell
- ✓ Windows native commands/APIs
- ✓ JSON configuration
- ✓ Terminal/TUI
- ✓ Optional .BAT launcher
- ✓ GitHub repository
- ✓ One-command execution

...

Then **v2** could move the performance-heavy scanner to Rust if PowerShell becomes the bottleneck.

I think this approach fits **HailUninstaller** much better: **download** → **run** → **scan** → **review** → **clean** → **exit**, with no software installation required.